

CPU Inference Benchmarking Results Summary

Matrix multiplication Paged KV cache IMBPS

Deveshi Singh October 2026

Main conclusion CPU inference performance depended on how work was grouped and how data was traversed. No single loop order or cache threshold predicted the fastest implementation. The strongest results came from reducing dispatch and allocation overhead, matching traversal to physical storage, and enabling IMBPS only when the active MLP workload was large enough.

Results at a glance

Benchmark	Observed result	Practical interpretation
Batched matrix multiplication	BMM remained faster after the nominal working set exceeded cache.	Dispatches, temporary allocations and copies still mattered.
Optimized GEMM loop orders	Six loop orders were usually within about 2 percent once they shared the same tiled SIMD microkernel.	Microkernel quality mattered more than the outer loop order.
Paged KV cache	Matched layout and traversal won; the preferred layout changed with physical block locality.	Choose layout using the expected fragmentation pattern.
IMBPS	Prefill gains were reproduced approximately. Decode shaped MLPs benefited only at high active row counts.	Select K empirically and bypass IMBPS for small decode workloads.

What changed across the experiments

- First, batched execution removed repeated framework dispatches and temporary result copies.
- Next, KV experiments separated physical layout from traversal order and exposed a fragmentation dependent crossover.
- Finally, IMBPS split the MLP expansion dimension to improve cache locality, but the best split still depended on kernel overhead and workload size.

Matrix multiplication findings

Batched execution remained faster

A loop of individual matrix multiplications performed the same useful FLOPs as one batched matrix multiplication, but it issued one operation per batch item, created temporary outputs, allocated a final stacked tensor and copied each temporary into it. Those costs remained visible even after the matrices exceeded the nominal cache capacity.

Implementation	Execution and memory behavior
Looped MM	Multiple aten::mm dispatches plus temporary C tensors and a final stacked C tensor
Batched MM	One aten::bmm dispatch and one final C allocation

The shared microkernel reduced loop order differences

Cache tiling plus SIMD register blocking was about 3.1 times faster than tiled-only multiplication in the earlier Apple ARM experiment. Once all six outer loop orders used that same microkernel, their geometric-mean performance was generally within about 2 percent of the best. The initial kij and ikj advantage was therefore workload-specific rather than universal.

Paged KV cache findings

The KV experiments separated physical storage from traversal. Matching the traversal order to the physical layout consistently improved locality. Which matching layout was best depended on whether the physical block table contained long contiguous runs or highly fragmented blocks.

Block table locality	Preferred organization	Observed result
Contiguous runs of at least 8 blocks	HBND with head-first traversal	At run length 512, HBND/BHND was 0.937 times
Run length 4	No clear winner	Effective tie
Runs of 1 or 2 blocks	BHND with block-first traversal	At run length 1, HBND/BHND was 1.059 times

A separate BF16 decode sweep found a repeatable region for block-major storage with block-first traversal. It won 39 of 48 MHA workloads with a median speedup of about 1.17 times, mainly at 40 to 72 MiB of logical K plus V traffic per call. The baseline had no repeatable wins in that region.

IMBPS prefill reproduction

IMBPS partitions the OPT-30B MLP expansion dimension and processes one block at a time. The reproduction used one EPYC 9654 socket with 96 physical cores and 384 MiB aggregate L3. The measured speedups were close to the paper, although K=8 slightly outperformed K=4 at the larger batches.

Batch	Paper K=4	Measured K=4	Fastest tested K
16	1.21 times	1.160 times	K=4 at 1.160 times
32	1.25 times	1.184 times	K=8 at 1.185 times
64	1.23 times	1.206 times	K=8 at 1.213 times

Cache fit did not guarantee speedup

For B=16 and sequence length 256, Equation 13 selected K=2 as the smallest power-of-two split whose modeled 364 MiB working set fit within 384 MiB L3. K=2 was statistically indistinguishable from TPP, and every larger split was slower. Cache residency alone was therefore not sufficient to predict runtime.

Backend	K	Modeled working set	Median time	Speedup over TPP
TPP	1	672 MiB	292.02 ms	1.000 times
IMBPS	2	364 MiB	292.40 ms	0.998 times CI 0.978 to 1.046
IMBPS	4	210 MiB	454.19 ms	0.645 times
IMBPS	8	133 MiB	330.55 ms	0.883 times
IMBPS	16	94.5 MiB	358.31 ms	0.818 times
IMBPS	32	75.25 MiB	613.19 ms	0.478 times

The equation predicted locality better than runtime

At sequence lengths 1024 and 1408, the Equation 13 candidate uniquely minimized L3 misses and DRAM traffic and maximized hit rate and measured arithmetic intensity. The next larger split had the lowest profiled runtime. This supports using the equation as a starting point followed by a small empirical search.

Sequence	Equation K	Best locality	Lowest profiled time
1024	16	K=16	K=32
1408	32	K=32	K=64

Decode shaped MLP pilot

The current extension evaluates the standalone OPT-30B MLP with one token per sequence. Active rows M represent the live decode batch; for speculative verification, the same M can represent batch multiplied by verified draft tokens. The pilot does not include attention, KV-cache access, proposal generation or token acceptance.

Preliminary decode shaped MLP crossover

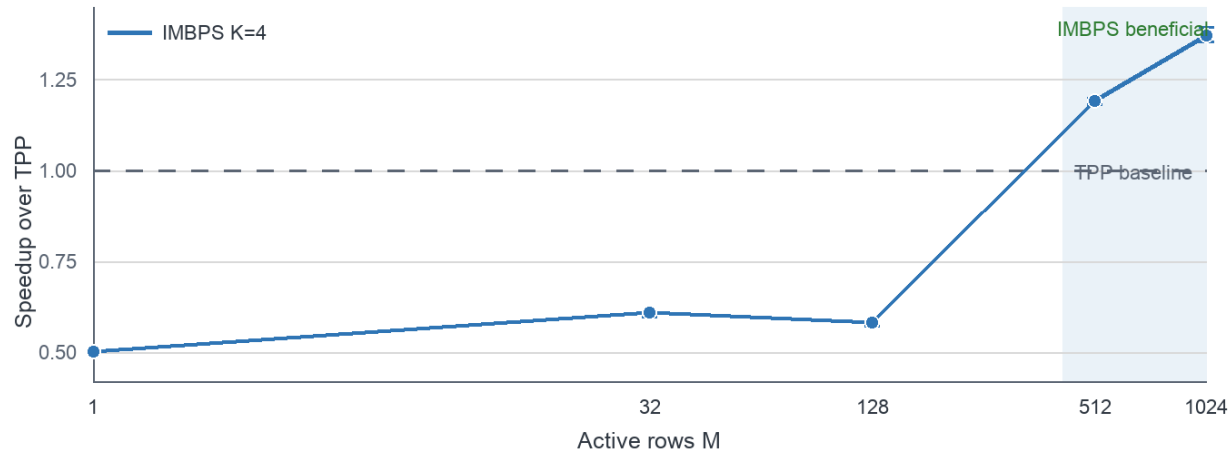


Figure 1 K=4 became beneficial between 128 and 512 active rows

Active rows	TPP	Best IMBPS K	IMBPS time	Paired speedup
1	1.711 ms	4	3.419 ms	0.501 times
32	3.591 ms	4	5.936 ms	0.609 times
128	8.236 ms	4	14.201 ms	0.581 times
512	39.001 ms	4	32.727 ms	1.192 times CI 1.182 to 1.198
1024	80.062 ms	4	58.420 ms	1.370 times CI 1.353 to 1.394

The pilot supports a conditional policy: use TPP for small active-row counts and consider IMBPS for highly batched decode or speculative verification. The full timing matrix is needed to locate the crossover more precisely. Counter evidence and end-to-end accepted-token throughput remain open.

Experimental conditions

Item	Earlier prefill experiments	Decode pilot
Processor placement	EPYC 9654 socket 0 96 cores 12 CCDs	Same
Cache model	384 MiB aggregate L3	96 MiB aggregate private L2
Model operator	OPT-30B MLP BF16 ReLU bias	Same
Software	PACE 1.0.0 PyTorch 2.7.0 CPU	Same
Memory settings	THP madvise no tcmalloc preload	THP always tcmalloc 2.18.1
Scheduling	Exclusive Slurm allocation one NUMA node	Same
Status	Completed timing and historical uProf mechanism runs	Preliminary three-round timing pilot

What is established and what remains open

- Established Batched dispatch reduces framework and allocation overhead beyond what a cache-size-only hypothesis predicts.
- Established KV layout should match traversal, and fragmentation changes the preferred layout.
- Established IMBPS reproduced substantial OPT-30B prefill gains on mn01, although the optimal K differed from the paper at larger batches.
- Preliminary Standalone decode-shaped MLPs benefited at 512 and 1024 active rows but regressed at 128 rows and below.
- Not yet established End-to-end decode or speculative-decode throughput, because attention, KV-cache work and acceptance behavior were excluded.
- Not yet established Parallel per-CCD IMBPS. Job 9300 only varied complete-CCD thread groups; it did not execute independent per-CCD splits.

Next experiments

Priority	Experiment	Decision it will support
1	Complete the full decode timing matrix over M=1 to 1024 and K=2 to 32	Choose a reliable active-row crossover and split policy
2	Add fixed-trace end-to-end normal and speculative decode	Test accepted-token throughput with attention and KV cache included
3	Implement parallel IMBPS with one worker group per CCD	Compare global splitting with CCD-local cache ownership and scaling
4	Repeat cache and traffic counters when amd_uncore access returns	Confirm whether L2 and DRAM traffic explain the decode crossover

Evidence

- [Table II timing summary](#)
- [Cache fit control](#)
- [Equation 13 uProf summary](#)
- [Decode pilot summary](#)
- [Decode experimental plan](#)